

UNIT - III Classification: Introduction – Statistical – Based Algorithms – Distance Based Algorithms – Decision.

4.1 INTRODUCTION

Classification is perhaps the most familiar and most popular data mining technique. Examples of classification applications include image and pattern recognition, medical diagnosis, loan approval, detecting faults in industry applications, and classifying financial market trends. Estimation and prediction may be viewed as types of classification. When someone estimates your age or guesses the number of marbles in a jar, these are actually classification problems. Prediction can be thought of as classifying an attribute value into one of a set of possible classes. It is often viewed as forecasting a continuous value, while classification forecasts a discrete value.

EXAMPLE 4.1

Teachers classify students as A, B, C, D, or F based on their marks. By using simple boundaries (60, 70, 80, 90), the following classification is possible:

$90 \leq \text{mark}$	A
$80 \leq \text{mark} < 90$	B
$70 \leq \text{mark} < 80$	C
$60 \leq \text{mark} < 70$	D
$\text{mark} < 60$	F

All approaches to performing classification assume some knowledge of the data. Often a training set is used to develop the specific parameters required by the technique. Training data consist of sample input data as well as the classification assignment for the data. Domain experts may also be used to assist in the process.

The classification problem is stated as shown in Definition 4.1:

DEFINITION 4.1. Given a database $D = \{t_1, t_2, \dots, t_n\}$ of tuples (items, records) and a set of classes $C = \{C_1, \dots, C_m\}$, the **classification problem** is to define a mapping $f : D \rightarrow C$ where each t_i is assigned to one class. A **class**, C_j , contains precisely those tuples mapped to it; that is, $C_j = \{t_i \mid f(t_i) = C_j, 1 \leq i \leq n, \text{ and } t_i \in D\}$.

Each tuple in the database is assigned to exactly one class. The classes that exist for a classification problem are indeed equivalence classes.

In actuality, the problem usually is implemented in two phases:

1. Create a specific model by evaluating the training data. This step has as input the training data (including defined classification for each tuple) and as output a definition of the model developed. The model created classifies the training data as accurately as possible.
2. Apply the model developed in step 1 by classifying tuples from the target database.

There are three basic methods used to solve the classification problem:

- Specifying boundaries. Here classification is performed by dividing the input space of potential database tuples into regions where each region is associated with one class.
- Using probability distributions. For any given class, C_j , $P(t_i | C_j)$ is the PDF for the class evaluated at one point, t_i . If a probability of occurrence for each class, $P(C_j)$ is known (perhaps determined by a domain expert), then $P(C_j)P(t_i | C_j)$ is used to estimate the probability that t_i is in class C_j .
- Using posterior probabilities. Given a data value t_i , we would like to determine the probability that t_i is in a class C_j . This is denoted by $P(C_j | t_i)$ and is called the posterior probability. One classification approach would be to determine the posterior probability for each class and then assign t_i to the class with the highest probability.

4.1.1 Issues in Classification

Missing Data. Missing data values cause problems during both the training phase and to the classification process itself. Missing values in the training data must be handled and may produce an inaccurate result. Missing data in a tuple to be classified must be able to be handled by the resulting classification scheme. There are many approaches to handling missing data:

- Ignore the missing data.
- Assume a value for the missing data. This may be determined by using some method to predict what the value could be.
- Assume a special value for the missing data. This means that the value of missing data is taken to be a specific value all of its own. Notice the similarity between missing data in the classification problem and that of nulls in traditional databases.

Measuring Performance. The performance of classification algorithms is usually examined by evaluating the accuracy of the classification. However, since classification is often a fuzzy problem, the correct answer may depend on the user. Traditional algorithm evaluation approaches such as determining the space and time overhead can be used, but these approaches are usually secondary. Classification accuracy is usually calculated by determining the percentage of tuples placed in the correct class. This ignores the fact that there also may be a cost associated with an incorrect assignment to the wrong class. This perhaps should also be determined.

STATISTICAL-BASED ALGORITHMS

4.2.1 Regression

Regression problems deal with estimation of an output value based on input values. When used for classification, the input values are values from the database D and the output values represent the classes. Regression can be used to solve classification problems, but it can also be used for other applications such as forecasting. Although not explicitly described in this text, regression can be performed using many different types of techniques, including NNs. In actuality, regression takes a set of data and fits the data to a formula.

EXAMPLE 4.2

Suppose that a graduate level abstract algebra class has 100 students. Sahana consistently outperforms the other students on exams. On the final exam, Sahana gets a grade of 99. The next highest grade is 75, with the range of grades being between 5 and 99. Sahana clearly is resented by the other students in the class because she does not perform at the same level they do. She “ruins the curve.” If we were to try to fit a model to the grades, this one outlier grade would cause problems because any model that attempted to include it would then not accurately model the remaining data.

We illustrate the process using a simple linear regression formula and assuming k points in our training sample. We thus have the following k formulas:

$$y_i = c_0 + c_1 x_{1i} + \epsilon_i, i = 1, \dots, k \quad (4.3)$$

With a simple linear regression, given an observable value (x_{1i}, y_i) , ϵ_i is the error, and thus the squared error technique introduced in Chapter 3 can be used to indicate the error. To minimize the error, a *method of least squares* is used to minimize the least squared error. This approach

finds coefficients c_0, c_1 so that the squared error is minimized for the set of observable values. The sum of the squares of the errors is

$$L = \sum_{i=1}^k \epsilon_i^2 = \sum_{i=1}^k (y_i - c_0 - c_1 x_{1i})^2 \quad (4.4)$$

Taking the partial derivatives (with respect to the coefficients) and setting equal to zero, we can obtain the *least squares estimates* for the coefficients, $\hat{c}_0$ and $\hat{c}_1$.

Regression can be used to perform classification using two different approaches:

1. **Division:** The data are divided into regions based on class.
2. **Prediction:** Formulas are generated to predict the output class value.

The first case views the data as plotted in an n -dimensional space without any explicit class values shown. Through regression, the space is divided into regions—one per class. With the second approach, a value for each class is included in the graph. Using regression, the formula for a line to predict class values is generated.

Example 4.3 illustrates the division process, while Example 4.4 illustrates the prediction process using the data from Table 4.1. For simplicity, we assume the training data include only data for short and medium people and that the classification is performed using the Output1 column values. If you extend this example to all three classes, you will see that it is a nontrivial task to use linear regression for classification. It also will become obvious that the result may be quite poor.

4.2.2 Bayesian Classification

Assuming that the contribution by all attributes are independent and that each contributes equally to the classification problem, a simple classification scheme called naive Bayes classification has been proposed that is based on Bayes rule of conditional probability. By analyzing the contribution of each “independent” attribute, a conditional probability is determined. A classification is made by combining the impact that the different attributes have on the prediction to be made. The approach is called “naive” because it assumes the independence between the various attribute values.

the naive Bayes algorithm first estimates the prior probability $P(C_j)$ for each class by counting how often each class occurs in the training data. For each attribute, x_i , the number of occurrences of each attribute value x_i can be counted to determine $P(x_i)$. Similarly, the probability $P(x_i | C_j)$ can be estimated by counting how often each value occurs in the class in the training data.

A tuple in the training data may have many different attributes, each with many values. This must be done for all attributes and all values of attributes. We then use these derived probabilities when a new tuple must be classified. This is why naive Bayes classification can be viewed as both a descriptive and a predictive type of algorithm.

EXAMPLE 4.5

Using the Output1 classification results for Table 4.1, there are four tuples classified as short, eight as medium, and three as tall. To facilitate classification, we divide the height attribute into six ranges:

$$(0, 1.6], (1.6, 1.7], (1.7, 1.8], (1.8, 1.9], (1.9, 2.0], (2.0, \infty)$$

Table 4.3 shows the counts and subsequent probabilities associated with the attributes. With these training data, we estimate the prior probabilities:

$$P(\text{short}) = 4/15 = 0.267, P(\text{medium}) = 8/15 = 0.533, \\ \text{and } P(\text{tall}) = 3/15 = 0.2$$

We use these values to classify a new tuple. For example, suppose we wish to classify $t = \langle \text{Adam}, M, 1.95 \text{ m} \rangle$. By using these values and the associated probabilities of gender and height, we obtain the following estimates:

$$P(t | \text{short}) = 1/4 \times 0 = 0 \\ P(t | \text{medium}) = 2/8 \times 1/8 = 0.031 \\ P(t | \text{tall}) = 3/3 \times 1/3 = 0.333$$

TABLE 4.3: Probabilities Associated with Attributes

Attribute	Value	Count			Probabilities		
		Short	Medium	Tall	Short	Medium	Tall
Gender	M	1	2	3	1/4	2/8	3/3
	F	3	6	0	3/4	6/8	0/3
Height	(0, 1.6]	2	0	0	2/4	0	0
	(1.6, 1.7]	2	0	0	2/4	0	0
	(1.7, 1.8]	0	3	0	0	3/8	0
	(1.8, 1.9]	0	4	0	0	4/8	0
	(1.9, 2]	0	1	1	0	1/8	1/3
	(2, ∞)	0	0	2	0	0	2/3

Combining these, we get

$$\text{Likelihood of being short} = 0 \times 0.267 = 0 \quad (4.9)$$

$$\text{Likelihood of being medium} = 0.031 \times 0.533 = 0.0166 \quad (4.10)$$

$$\text{Likelihood of being tall} = 0.33 \times 0.2 = 0.066 \quad (4.11)$$

We estimate $P(t)$ by summing up these individual likelihood values since t will be either short or medium or tall:

$$P(t) = 0 + 0.0166 + 0.066 = 0.0826 \quad (4.12)$$

Finally, we obtain the actual probabilities of each event:

$$P(\text{short} | t) = \frac{0 \times 0.0267}{0.0826} = 0 \quad (4.13)$$

$$P(\text{medium} | t) = \frac{0.031 \times 0.533}{0.0826} = 0.2 \quad (4.14)$$

$$P(\text{tall} | t) = \frac{0.333 \times 0.2}{0.0826} = 0.799 \quad (4.15)$$

Therefore, based on these probabilities, we classify the new tuple as tall because it has the highest probability.

The naive Bayes approach has several advantages. First, it is easy to use. Second, unlike other classification approaches, only one scan of the training data is required. The naive Bayes approach can easily handle missing values by simply omitting that probability when calculating the likelihoods of membership in each class. In cases where there are simple relationships, the technique often does yield good results. Although the naive Bayes approach is straightforward to use, it does not always yield satisfactory results. First, the attributes usually are not independent. We could use a subset of the attributes by ignoring any that are dependent on others. The technique does not handle continuous data. Dividing the continuous values into ranges could be used to solve this problem, but the division of the domain into ranges is not an easy task, and how this is done can certainly impact the results.

4.3 DISTANCE-BASED ALGORITHMS

Each item that is mapped to the same class may be thought of as more similar to the other items in that class than it is to the items found in other classes. Therefore, similarity (or distance) measures may be used to identify the “aliqueness” of different items in the database. Using a similarity measure for classification where the classes are pre defined is somewhat simpler than using a similarity measure for clustering where the classes are not known in advance. Again, think of the IR example. Each IR query provides the class definition in the form of the IR query itself. So the classification problem then becomes one of determining similarity not among all tuples in the database but between each tuple and the query. This makes the problem an $O(n)$ problem rather than an $O(n^2)$ problem.

Simple Approach

Simple Approach Using the IR approach, if we have a representative of each class, we can perform classification by assigning each tuple to the class to which it is most similar. We assume here that each tuple, t_i , in the database is defined as a vector $t_{i1}, t_{i2}, \dots, t_{ik}$ of numeric values. Likewise, we assume that each class C_j is defined by a tuple $C_{j1}, C_{j2}, \dots, C_{jk}$ of numeric values.

The classification problem is then restated in Definition 4.2.

DEFINITION 4.2. Given a database $D = \{t_1, t_2, \dots, t_n\}$ of tuples where each tuple $t_i = t_{i1}, t_{i2}, \dots, t_{ik}$ contains numeric values and a set of classes $C = \{C_1, \dots, C_m\}$ where each class $C_j = C_{j1}, C_{j2}, \dots, C_{jk}$ has numeric values, the classification problem is to assign each t_i to the class C_j such that $(t_i, C_j) \geq \text{sim}(t_i, C_l) \forall C_l \in C \text{ where } C_l \neq C_j$.

To calculate these similarity measures, the representative vector for each class must be determined. Referring to the three classes in Figure 4.1(a), we can determine a representative for each class by calculating the center of each region. Thus class A is represented by 4, 7.5, class B by 2, 2.5, and class C by 6, 2.5. A simple classification technique, then, would be to place each item in the class where it is most similar (closest) to the center of that class. The representative for the class may be found in other ways. For example, in pattern recognition problems, a predefined pattern can be used to represent each class. Once a similarity measure is defined, each item to be classified will be compared to each predefined pattern. The item will be placed in the class with the largest similarity value. Algorithm 4.1 illustrates a straightforward distance-based approach assuming that each class, c_i , is represented by its center or centroid. In the algorithm we use c_i to be the center for its class. Since each tuple must be compared to the center for a class and there are a fixed (usually small) number of classes, the complexity to classify one tuple is $O(n)$.

ALGORITHM 4.1

Input:

$c_1, \dots, c_m$ //Centers for each class
 t //Input tuple to classify

Output:

c //Class to which t is assigned

Simple distance-based algorithm

$\text{dist} = \infty$;

for $i := 1$ to m **do**

if $\text{dis}(c_i, t) < \text{dist}$, **then**

$c = i$;

$\text{dist} = \text{dist}(c_i, t)$;

Figure 4.8 illustrates the use of this approach to perform classification using the data found in Figure 4.1. The three large dark circles are the class representatives for the three classes. The dashed lines show the distance from each item to the closest center.

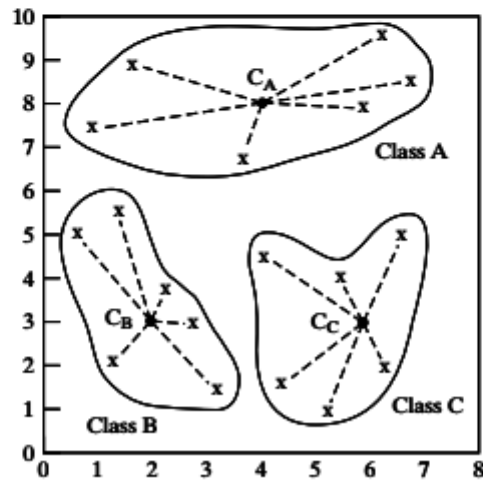


FIGURE 4.8: Classification using simple distance algorithm.

4.3.2 K Nearest Neighbors

One common classification scheme based on the use of distance measures is that of the K nearest neighbors (KNN). The KNN technique assumes that the entire training set includes not only the data in the set but also the desired classification for each item. In effect, the training data become the model. When a classification is to be made for a new item, its distance to each item in the training set must be determined. Only the K closest entries in the training set are considered further. The new item is then placed in the class that contains the most items from this set of K closest items. Figure 4.9 illustrates the process used by KNN. Here the points in the training set are shown and $K = 3$. The three closest items in the training set are shown; t will be placed in the class to which most of these are members.

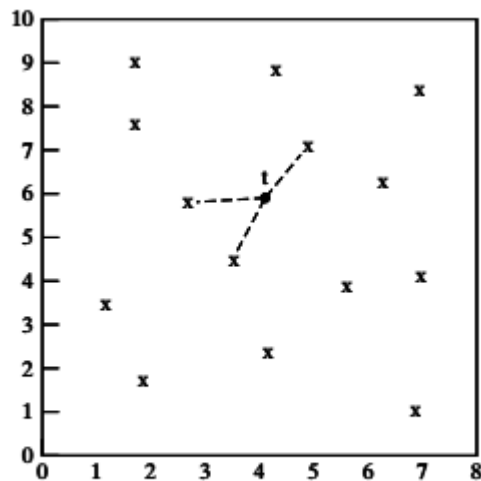


FIGURE 4.9: Classification using KNN.

Algorithm 4.2 outlines the use of the KNN algorithm. We use T to represent the training data. Since each tuple to be classified must be compared to each element in the training data, if there are q elements in the training set, this is $O(q)$. Given n elements to be classified, this becomes an $O(nq)$ problem. Given that the training data are of a constant size (although perhaps quite large), this can then be viewed as an $O(n)$ problem.

ALGORITHM 4.2

Input:

T //Training data
 K //Number of neighbors
 t //Input tuple to classify

Output:

c //Class to which t is assigned

KNN algorithm:

```

    //Algorithm to classify tuple using KNN
 $N = \emptyset$ ;
    //Find set of neighbors,  $N$ , for  $t$ 
    for each  $d \in T$  do
        if  $|N| \leq K$ , then
             $N = N \cup \{d\}$ ;
        else
            if  $\exists u \in N$  such that  $\text{sim}(t, u) \leq \text{sim}(t, d)$ , then

```



```

begin
     $N = N - \{u\};$ 
     $N = N \cup \{d\};$ 
end
//Find class for classification
c = class to which the most  $u \in N$  are classified;

```

Example 4.6 illustrates this technique using the sample data from Table 4.1. The KNN technique is extremely sensitive to the value of K . A rule of thumb is that $K \leq \sqrt{\text{number of training items}}$ [KLR⁺98]. For this example, that value is 3.46. Commercial algorithms often use a default value of 10.

EXAMPLE 4.6

Using the sample data from Table 4.1 and the Output1 classification as the training set output value, we classify the tuple $\langle \text{Pat}, F, 1.6 \rangle$. Only the height is used for distance calculation so that both the Euclidean and Manhattan distance measures yield the same results; that is, the distance is simply the absolute value of the difference between the values. Suppose that $K = 5$ is given. We then have that the K nearest neighbors to the input tuple are $\{\langle \text{Kristina}, F, 1.6 \rangle, \langle \text{Kathy}, F, 1.6 \rangle, \langle \text{Stephanie}, F, 1.7 \rangle, \langle \text{Dave}, M, 1.7 \rangle, \langle \text{Wynette}, F, 1.75 \rangle\}$. Of these five items, four are classified as short and one as medium. Thus, the KNN will classify Pat as short.
